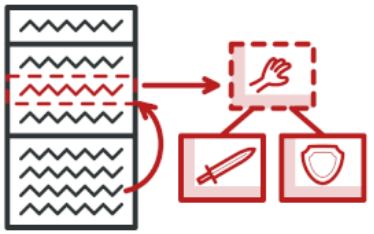
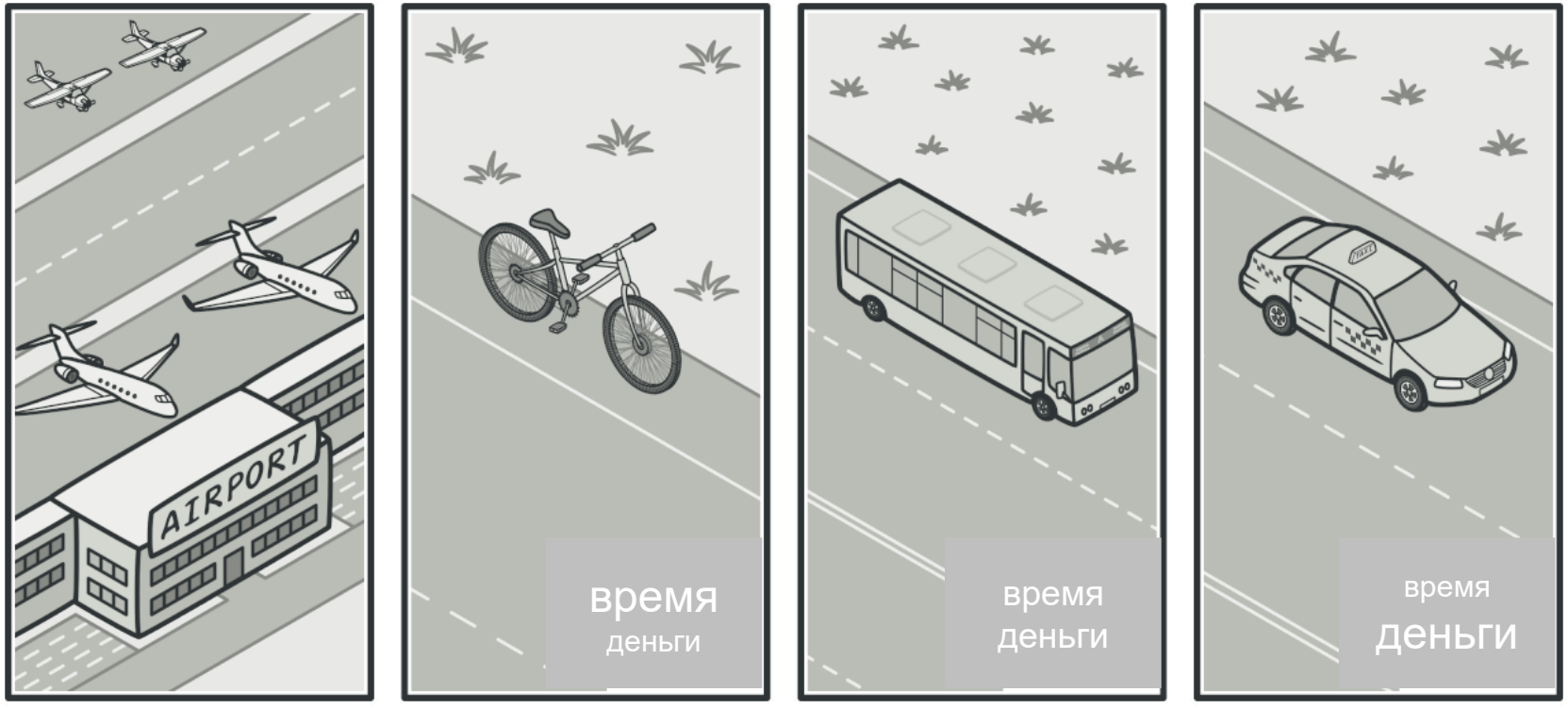
Поведенческие паттерны. Стратегия



Паттерн Стратегия
Strategy

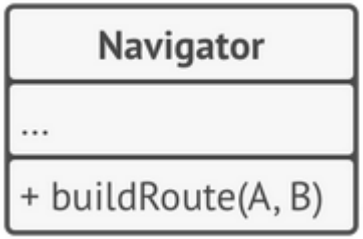


Стратегия — это поведенческий паттерн проектирования, который определяет семейство схожих алгоритмов и помещает каждый из них в собственный класс, после чего алгоритмы можно взаимозаменять во время исполнения программы.

Другие объекты содержат ссылку на объект-стратегию и делегируют ей работу. Программа может подменить этот объект другим, если требуется иной способ решения задачи.

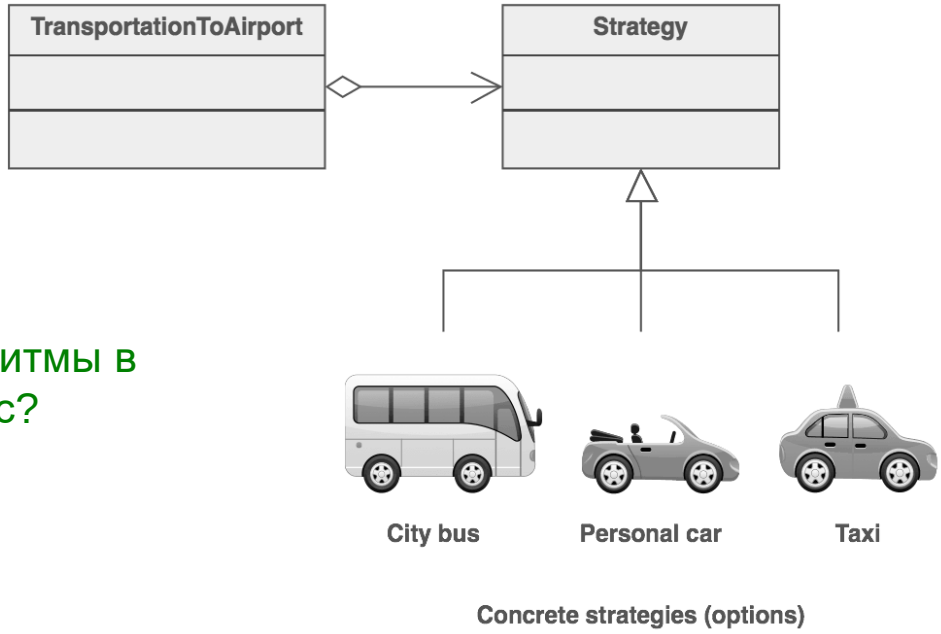
Проблема

Класс навигатора «все-в-одном»



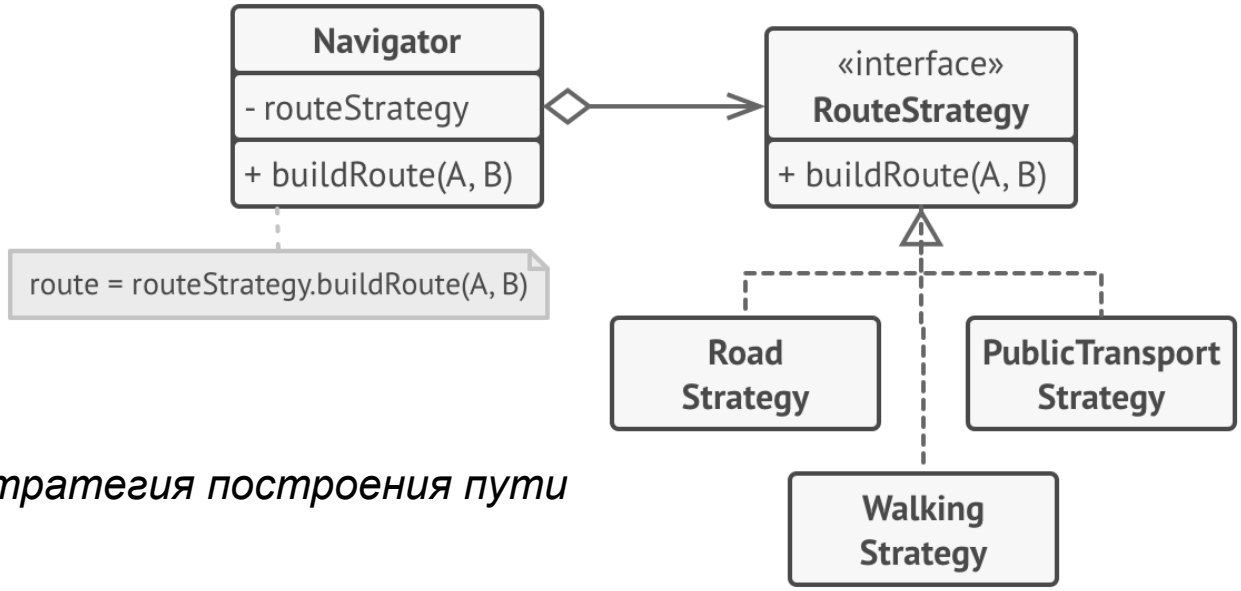
Проблемы с поддержкой и расширением
God object
Единство ответственности

Выделить алгоритмы в отдельный класс?



Решение

Определить семейство схожих алгоритмов, которые часто изменяются или расширяются, и вынести их в собственные классы, называемые ***стратегиями***.



Стратегия построения пути

Класс Navigator играет роль контекста, ссылаясь на одну из стратегий и делегируя ей выполнение работы.

Чтобы сменить алгоритм, достаточно подставить в контекст другой объект-стратегию.

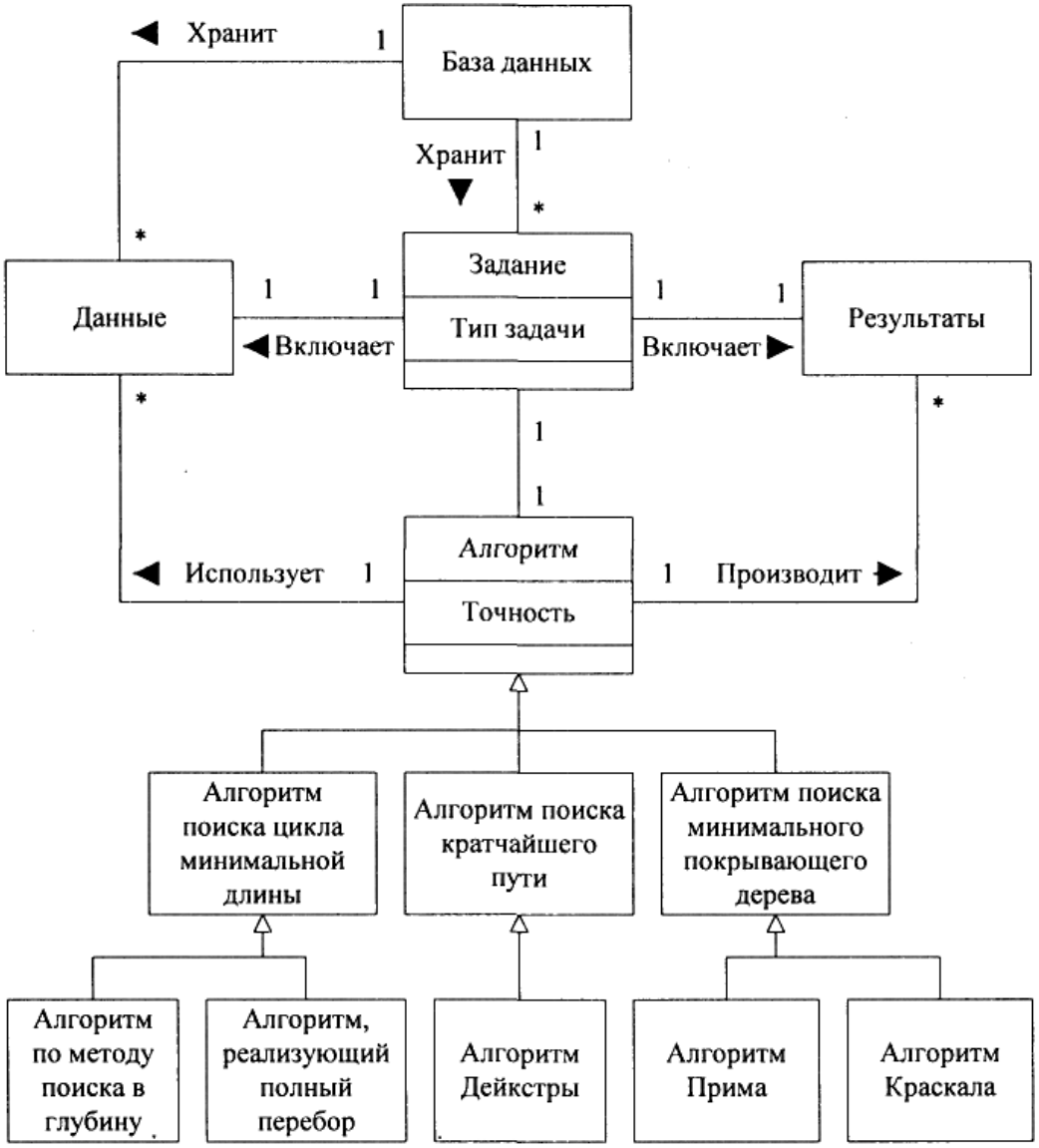
Все стратегии должны иметь общий интерфейс.

Используя этот интерфейс, контекст будет независимым от конкретных классов стратегий.

Можно изменять и добавлять новые виды алгоритмов, не затрагивая код контекста.

Решение

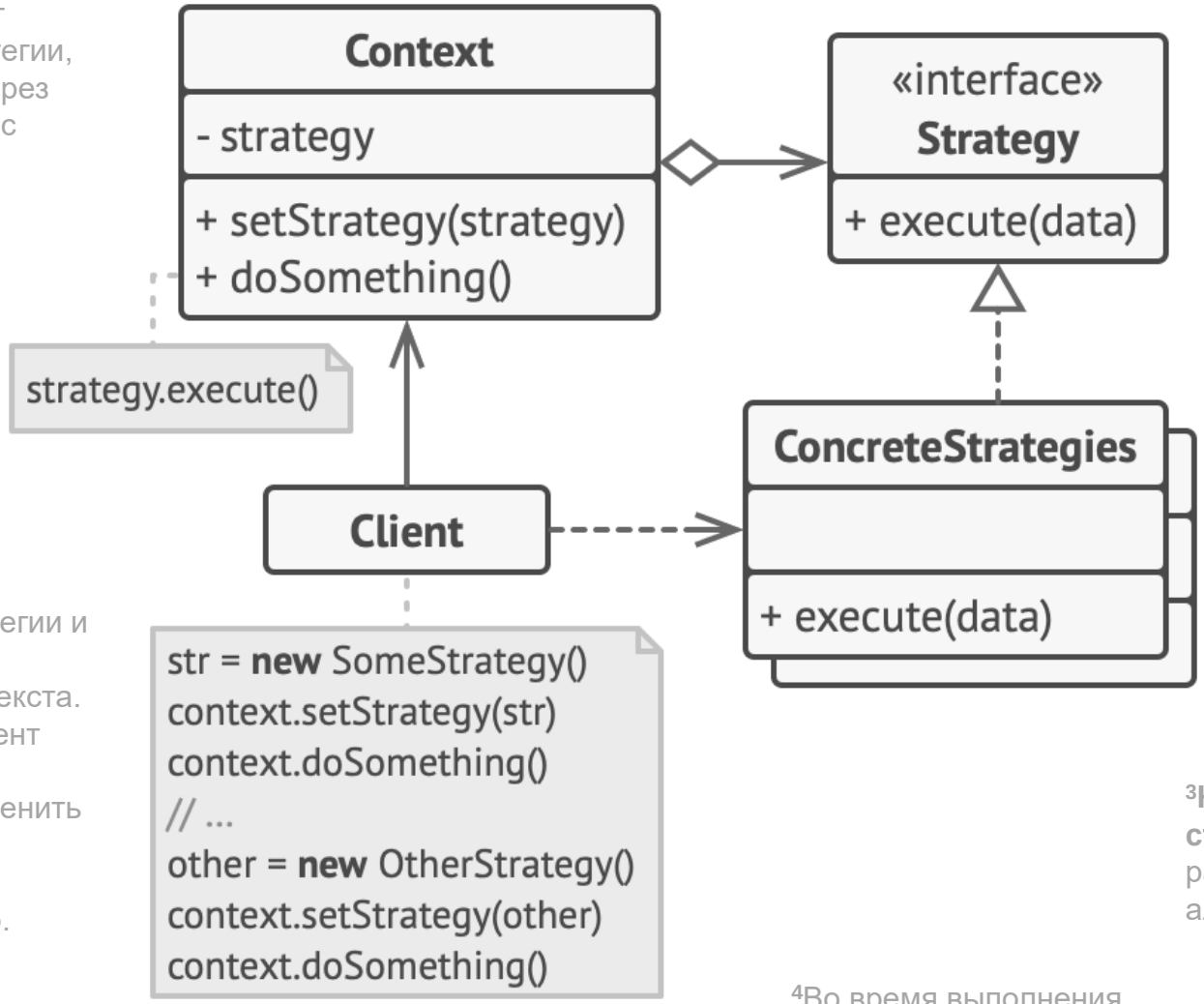
Определить семейство схожих алгоритмов, которые часто изменяются или расширяются, и вынести их в собственные классы, называемые ***стратегиями***.



Решение оптимизационной задачи

Структура паттерна

[1]**Контекст** хранит ссылку на объект конкретной стратегии, работая с ним через общий интерфейс стратегий.



[2]**Стратегия** определяет интерфейс, общий для всех вариаций алгоритма. Контекст использует этот интерфейс для вызова алгоритма.

Для контекста неважно, какая именно вариация алгоритма будет выбрана, так как все они имеют одинаковый интерфейс.

[5]Клиент должен создать объект конкретной стратегии и передать его в конструктор контекста. Кроме этого, клиент должен иметь возможность заменить стратегию динамически, используя сеттер.

[3]**Конкретные стратегии** реализуют различные вариации алгоритма.

[4]Во время выполнения программы контекст получает вызовы от клиента и делегирует их объекту конкретной стратегии.

Пример

Стратегия для калькулятора

```
// Общий интерфейс всех стратегий.
interface Strategy is
    method execute(a, b)

// Каждая конкретная стратегия реализует общий интерфейс своим
// способом.
class ConcreteStrategyAdd implements Strategy is
    method execu
        return a

class ConcreteSt
    method execu
        return a

class ConcreteSt
    method execu
        return a
```

```
// Контекст всегда работает со стратегиями через общий
// интерфейс. Он
class Context is
    private strat

    method setStr
        this.stra

    method execut
        return st
```

```
class ExampleApplication is
    method main() is
        // 1. Создать объект контекста.
        // 2. Получить первое число (n1).
        // 3. Получить второе число (n2).
        // 4. Получить желаемую операцию.
        // 5. Затем, выбрать стратегию:

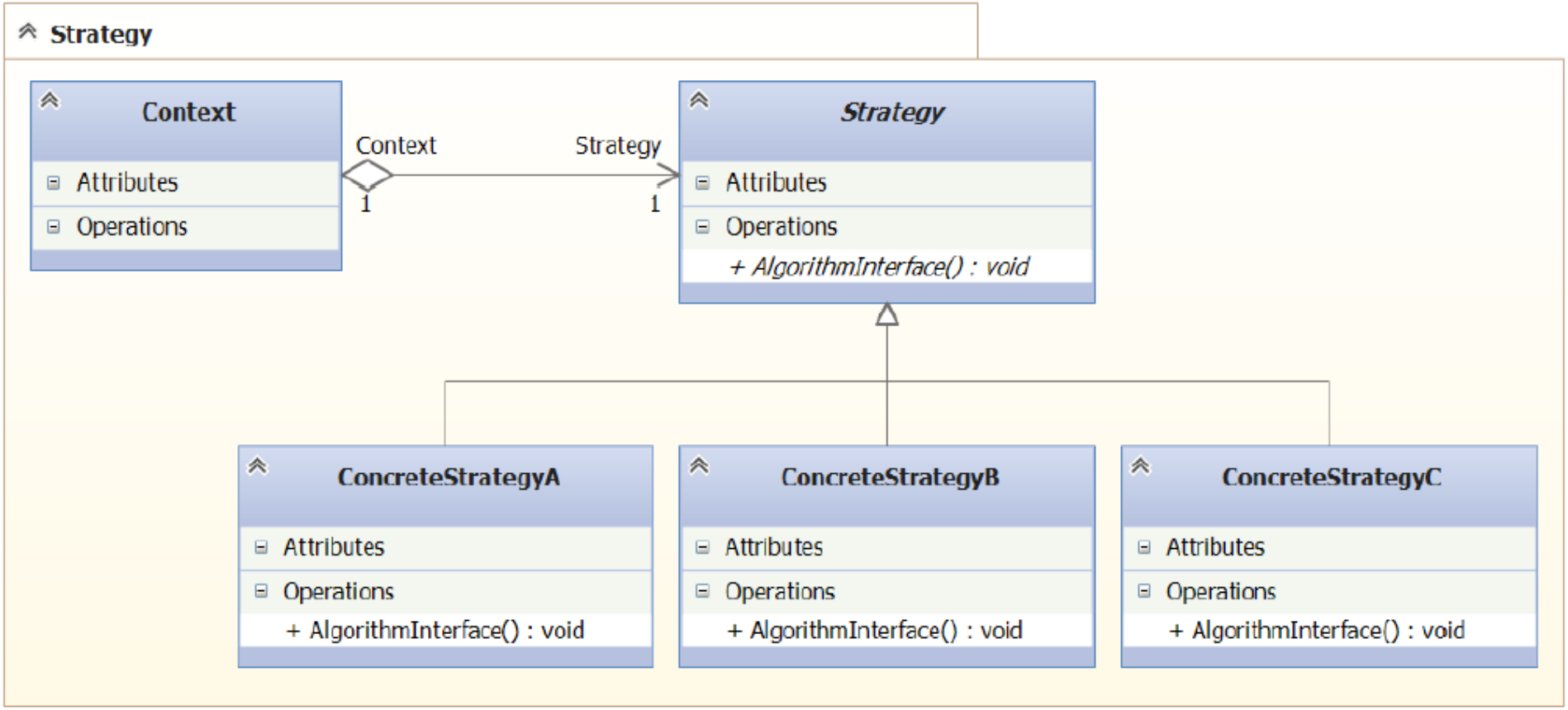
        if (action == addition) then
            context.setStrategy(new ConcreteStrategyAdd())

        if (action == subtraction) then
            context.setStrategy(new ConcreteStrategySubtract())

        if (action == multiplication) then
            context.setStrategy(new ConcreteStrategyMultiply())

        // 6. Выполнить операцию с помощью стратегии:
        result = context.executeStrategy(n1, n2)
```

Структура паттерна UML



Структура паттерна на языке C#

```
class Program
{
    static void Main()
    {
        Context context;
        context = new Context(new ConcreteStrategyA());
        context.ContextInterface();
        context = new Context(new ConcreteStrategyB());
        context.ContextInterface();
        context = new Context(new ConcreteStrategyC());
        context.ContextInterface();
    }
}
```

`Context` - **Контекст**:
Объект класса `Context` конфигурируется объектом класса `ConcreteStrategy`.

```
class Context
{
    Strategy strategy;

    public Context(Strategy strategy)
    {
        this.strategy = strategy;
    }

    public void ContextInterface()
    {
        strategy.AlgorithmInterface();
    }
}
```

`Strategy` - **Стратегия**:
Предоставляет интерфейс для реализаций конкретных стратегий

```
abstract class Strategy
{
    public abstract void AlgorithmInterface();
}
```

`ConcreteStrategy` - **Конкретная стратегия**:
Классы конкретных стратегий реализуют интерфейс, предоставляемый классом `Strategy`. Реализации интерфейса, предоставляемого классом `Strategy` представляют собой алгоритмы (заданного вида деятельности) направленные на достижение определенной цели.

```
class ConcreteStrategyA : Strategy
{
    public override void AlgorithmInterface()
    {
        Console.WriteLine("ConcreteStrategyA");
    }
}
```

```
class ConcreteStrategyB : Strategy
{
    public override void AlgorithmInterface()
    {
        Console.WriteLine("ConcreteStrategyB");
    }
}
```

```
class ConcreteStrategyC : Strategy
{
    public override void AlgorithmInterface()
    {
        Console.WriteLine("ConcreteStrategyC");
    }
}
```

Отношения между участниками

Отношения между классами

- Конкретные классы ConcreteStrategy связаны связями отношения наследования с абстрактным классом Strategy.
- Конкретный класс Context связан связью отношения агрегации с абстрактным классом Strategy.
- Отношения между объектами
- Классы Context и Strategy взаимодействуют друг с другом для реализации техники абстрагирования вариантов использования конкретных стратегий. Контекст может передавать определенной стратегии (ConcreteStrategy) все необходимые данные которые будут использоваться алгоритмом из этой стратегии
- Обычно клиент создает экземпляр класса Context и конфигурирует его экземпляром класса одной из конкретных стратегий (ConcreteStrategy), после чего клиент общается только с контекстом, который переадресует запросы клиентов объекту стратегии.

Применение

1. Когда нужно использовать разные вариации какого-то алгоритма внутри одного объекта.

Стратегия позволяет варьировать поведение объекта во время выполнения программы, подставляя в него различные объекты-поведения.

2. Когда есть множество похожих классов, отличающихся только некоторым поведением.

Стратегия позволяет вынести отличающееся поведение в отдельную иерархию классов, а затем свести первоначальные классы к одному, сделав поведение этого класса настраиваемым.

3. Когда необходимо скрыть детали реализации алгоритмов для других классов.

Стратегия позволяет изолировать код, данные и зависимости алгоритмов от других объектов, скрыв эти детали внутри классов-стратегий.

4. Когда различные вариации алгоритмов реализованы в виде сложного условного оператора. Каждая ветка такого оператора представляет собой вариацию алгоритма.

Стратегия помещает каждую ветку такого оператора в отдельный класс-стратегию. Затем контекст получает определённый объект-стратегию от клиента и делегирует ему работу. Если вдруг понадобится сменить алгоритм, в контекст можно подать другую стратегию.

Шаги реализации

1. **Определить алгоритм, который подвержен частым изменениям**. Также подойдёт алгоритм, имеющий несколько вариаций, которые выбираются во время выполнения программы.

2. **Создать интерфейс стратегий, описывающий этот алгоритм**. Он должен быть общим для всех вариантов алгоритма.

3. **Поместить вариации алгоритма в собственные классы**, которые реализуют этот интерфейс.

4. **В классе контекста создать поле для хранения ссылки на текущий объект-стратегию, а также метод для её изменения**. Проверить, что контекст работает с этим объектом только через общий интерфейс стратегий.

5. **Клиенты контекста должны подавать в него** соответствующий объект-стратегию, когда хотят, чтобы контекст вёл себя определённым образом.

Преимущества и недостатки

+ Динамическая смена алгоритмов.

+ Изолирует код и данные алгоритмов от остальных классов.

+ Уход от наследования к делегированию.

+ Реализует принцип открытости/закрытости.

- Усложняет программу за счёт дополнительных классов.

- Клиент должен знать, в чём состоит разница между стратегиями, чтобы выбрать подходящую.

Различия между паттернами	
Стратегия	**Состояние**
• позволяет выбрать клиенту другое поведение; • клиент управляет изменением в поведении, предоставляя различные стратегии к контексту;	• позволяет объекту вести себя по-разному в разном состоянии; • изменением состояния управляет контекст или текущее состояние объекта(если это реализовано);